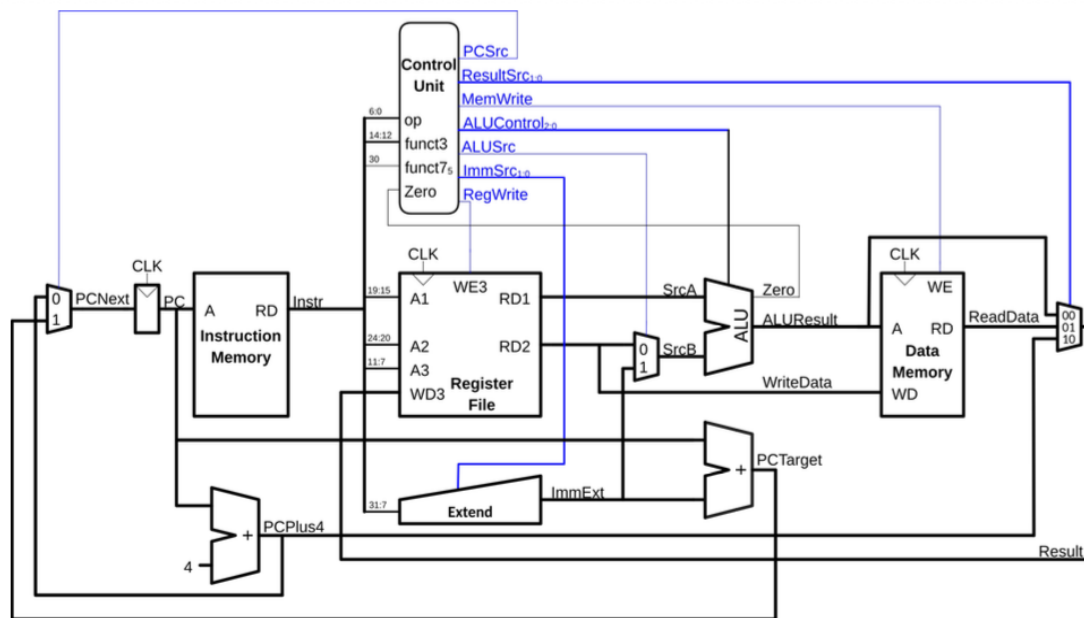


ERA Tutorium 12

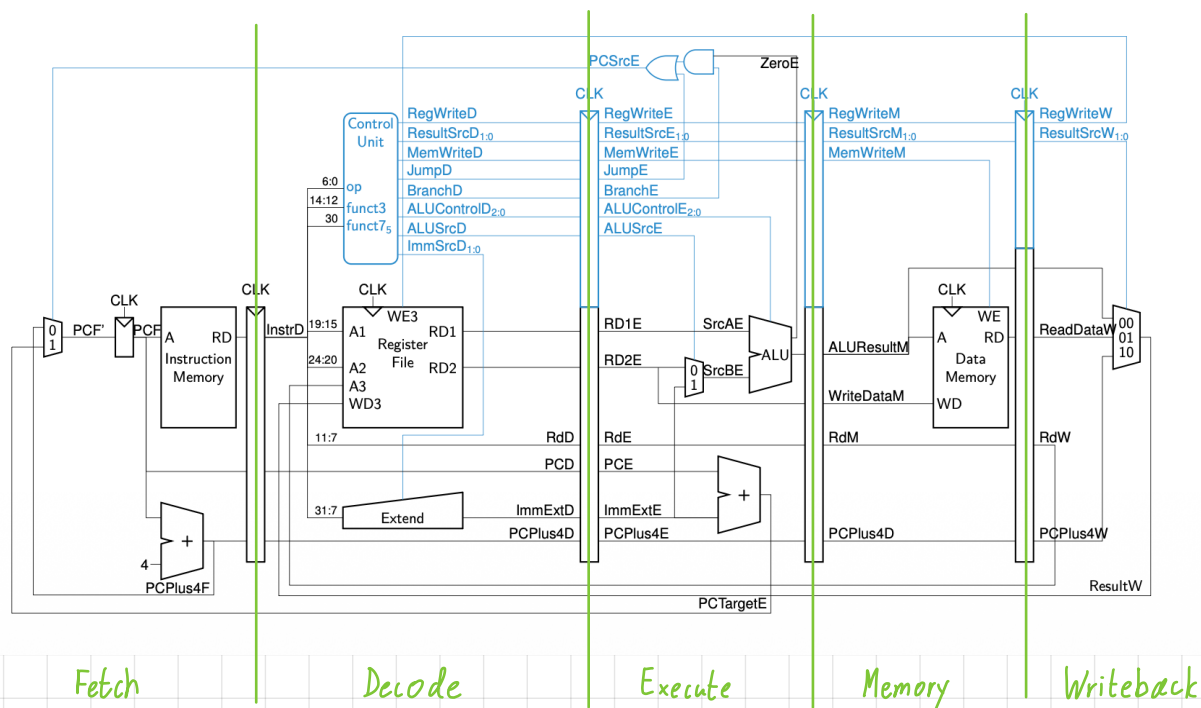
Leopold Cario	<u>Termin:</u>	<u>Raum:</u>	<u>Zulip:</u>
	· Mittwoch 10:00	03.13.010	ERA Tutorium - Mi - 1000 - 1
	· Donnerstag 12:00	00.08.059	ERA Tutorium - Do - 1200 - 1

Pipelining:

Single-Cycle RISC-V



Pipelined Prozessor: Schaltbild



	Fetch	Decode	Execute	Memory	Writeb.
1	i1				
2	i2	i1			
3	i3	i2	i1		
4	i4	i3	i2	i1	
5	i5	i4	i3	i2	i1

- **RAW** (Read-After-Write): potentiell ein Problem, falls schreibende Instruktion noch nicht zurückgeschrieben hat
- **WAR** (Write-After-Read): Reihenfolge der betroffenen Instruktionen darf nicht vertauscht werden
- **WAW** (Write-After-Write): Reihenfolge der betroffenen Instruktionen darf nicht vertauscht werden

```
lw t0, 0(a1)
add t2, t0, s5
```

RAW-Abhängigkeit bzgl. t0

```
xor a0, a2, a1
sub a2, a4, a5
```

WAR-Abhängigkeit bzgl. a2

```
and s0, s1, s2
lw s0, 0(s3)
```

WAW-Abhängigkeit bzgl. s0

Pipelinekonflikte

Data Hazards (Datenkonflikte¹)

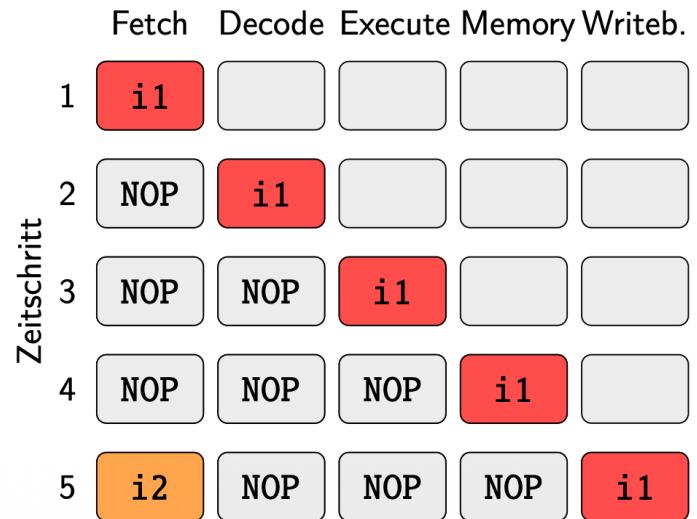
- können nur bei RAW auftreten²
- abhängige Instruktion in Decode, aber Ergebnis noch nicht zurückgeschrieben

Control Hazards (Steuerkonflikte)

- Kontrollflussänderungen durch branches/jumps
- falsche Sprungentscheidung: falsche Instruktionen in Pipeline geladen

Mindestens **3 Befehle** zwischen zwei
Instruktionen mit RAW-Abhängigkeit:

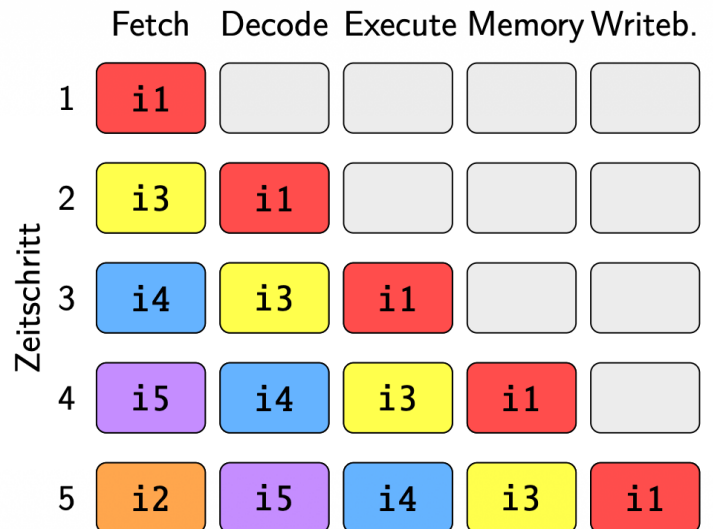
- NOPs (Stalling)



Konflikt zwischen i1 und i2

Mindestens **3 Befehle** zwischen zwei
Instruktionen mit RAW-Abhängigkeit:

- NOPs (Stalling)
- Befehlsumordnung (ohne Änderung der Semantik)



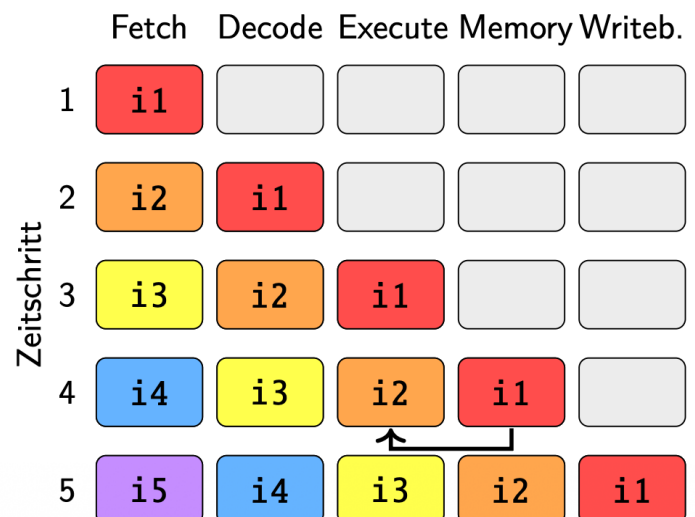
Konflikt zwischen i1 und i2

Mindestens **3 Befehle** zwischen zwei
Instruktionen mit RAW-Abhängigkeit:

- NOPs (Stalling)
- Befehlsumordnung (ohne Änderung der Semantik)

Alternativ:

- Forwarding: noch nicht
zurückgeschriebenes Ergebnis kann von
einer Stage in eine andere geleitet werden

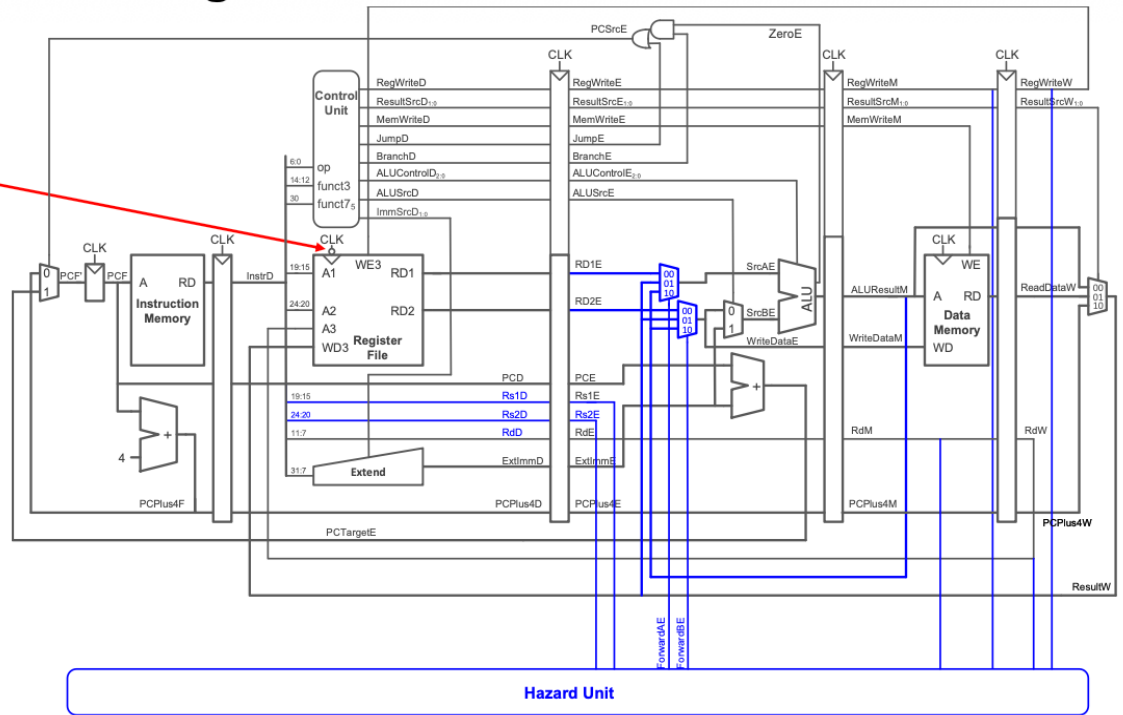


Konflikt zwischen i1 und i2

Hazard Unit: Forwarding

```
S1 addi s8, s4, 5
S2 sub s2, s8, s3
S3 or s9, t6, s8
S4 and s7, s8, t2
```

- Hazard Unit prüft ob Quellregister des Befehls in Execute mit dem Zielregister in Memory **oder Writeback** übereinstimmt
- Muss für beide Register geprüft werden
- Kein Forwarding von Writeback zu Decode nötig



71

Lösung von Konflikten: Control Hazards

Mindestens **2 Befehle** zwischen Sprungentscheidung und möglicherweise falsch geladener Instruktion:

- NOPs (Stalling)

Alternativ:

- Flushing: Falls Vorhersage der branch prediction falsch war, müssen geladene Instruktionen aus Pipeline entfernt werden

	Fetch	Decode	Execute	Memory	Writeb.
1	i1				
2	i2	i1			
3	i3	i2	i1		
4	i22	NOP	NOP	i1	
5	i23	i22	NOP	NOP	i1

Branch durch i1

Lehrstuhl für
Rechnerarchitektur & Parallele Systeme
Prof. Dr. Martin Schulz
Dominic Prinz
Jakob Schäffeler

Lehrstuhl für
Design Automation
Prof. Dr.-Ing. Robert Wille
Stefan Engels
Benjamin Hien

Einführung in die Rechnerarchitektur

Wintersemester 2025/2026

Übungsblatt 12: Pipelining

19.01.2026 – 23.01.2026

1 Pipelining Speedup

Element	Parameter	Delay (ps)
Register read	t_{RegRead}	40
Register setup	t_{RegSetup}	50
Multiplexer	t_{mux}	30
AND-OR gate	$t_{\text{AND-OR}}$	20
ALU	t_{ALU}	120
Decoder (Control Unit)	t_{dec}	25
Extend unit	t_{ext}	35
Memory read	t_{MemRead}	200
Memory setup	t_{MemSetup}	150
Register file read	t_{RFRead}	100
Register file setup	t_{RFSetup}	60

Tabelle 1: Propagation Delays

Nimm an, dass die Komponenten des RISC-V Prozessors die in Tabelle 1 angegebenen Verzögerungen haben (eine Verzögerung gilt zwischen jedem Eingang und Ausgang der Komponente). Für nicht gelistete Komponenten soll eine Verzögerung von 0 ps angenommen werden.

- a) Zeichne die *kritischen Pfade* aller Pipelining-Stufen des RISC-V-Prozessors in Abbildung 1 ein und bestimme deren Länge (in ps).

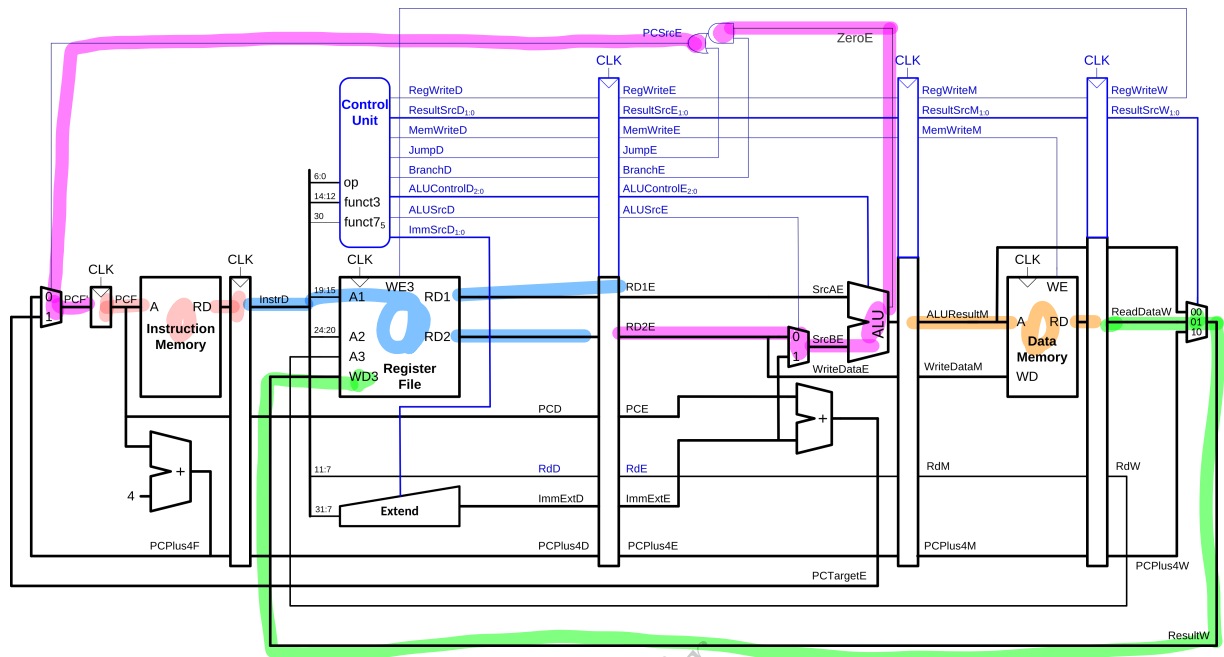
Fetch: $t_{\text{RegRead}} + t_{\text{MemRead}} + t_{\text{RegSetup}} = 290\text{ps}$

Decode: $t_{\text{RegRead}} + t_{\text{RFRead}} + t_{\text{RegSetup}} = 190\text{ps}$

Execute: $t_{\text{RegRead}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{AND-OR}} + t_{\text{mux}} + t_{\text{RegSetup}} = 290\text{ps}$

Memory: $t_{\text{RegRead}} + t_{\text{MemRead}} + t_{\text{RegSetup}} = 290\text{ps}$

Writeback: $t_{\text{RegRead}} + t_{\text{mux}} + t_{\text{RFSetup}} = 130\text{ps}$



- b) Bestimme die minimale Länge des Taktzyklus für den RISC-V Prozessors mit fünfstufiger Pipeline (Abbildung 1). Welcher Speed-up kann im Vergleich zum Prozessor ohne Pipeline (Abbildung 3, vgl. kombinatorischer Prozessor vom letzten Übungszettel) theoretisch erreicht werden? Warum ist dieser Speed-up nicht gleich der Anzahl der Pipeline-Stufen?

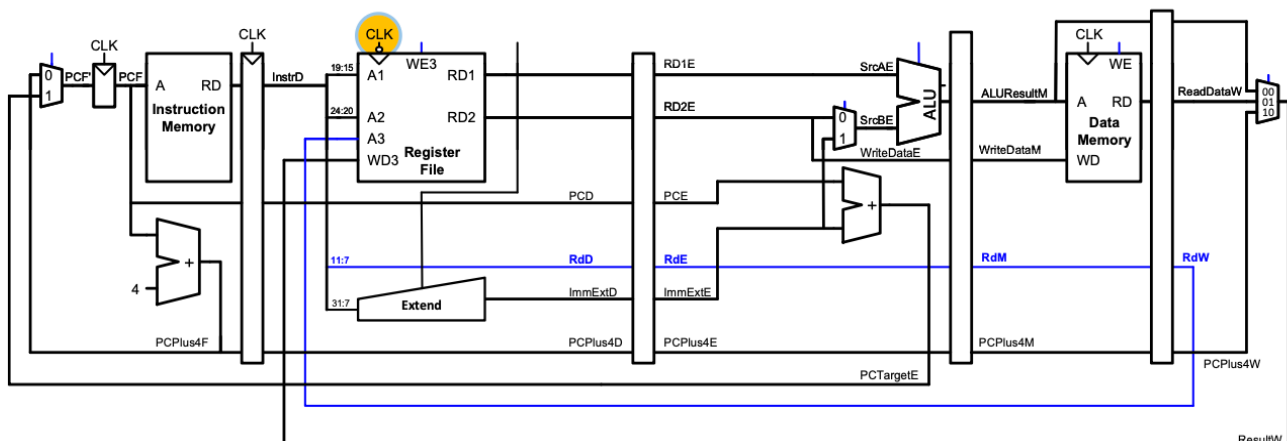
290ps

$$\frac{t_{\text{single cycle}}}{t_{\text{pipelined}}} = \frac{750\text{ps}}{290\text{ps}} = \underline{2,59}$$

Pipelined RISC-V: Korrigierter Datenpfad



- **Optimierung:** Registerbank wird zur **fallenden Flanke** von CLK geschrieben (damit Schreiben in erster Hälfte des Takts und Lesen des geschriebenen Wertes in zweiter Hälfte des Takts möglich).
- Dadurch müssen sich zwischen zwei direkt aufeinander folgenden Befehlen mit RAW-Abhängigkeit nur noch **mindestens 2 andere Befehle** befinden



- c) Wie ändert sich die Taktlänge wenn die Registerbank (wie in der Vorlesung vorgestellt) optimiert wird und nun bei fallender Flanke beschrieben wird? Damit ist das Schreiben in der ersten Hälfte des Takts möglich. Schreiben und Lesen können daher in einem Takt durchgeführt werden, jedoch kann die Registerbank somit erst in der zweiten Hälfte des Taktes gelesen werden.

$$\text{Decode: } (t_{RFRead} + t_{RegSetup}) \cdot 2 = \underline{300ps}$$

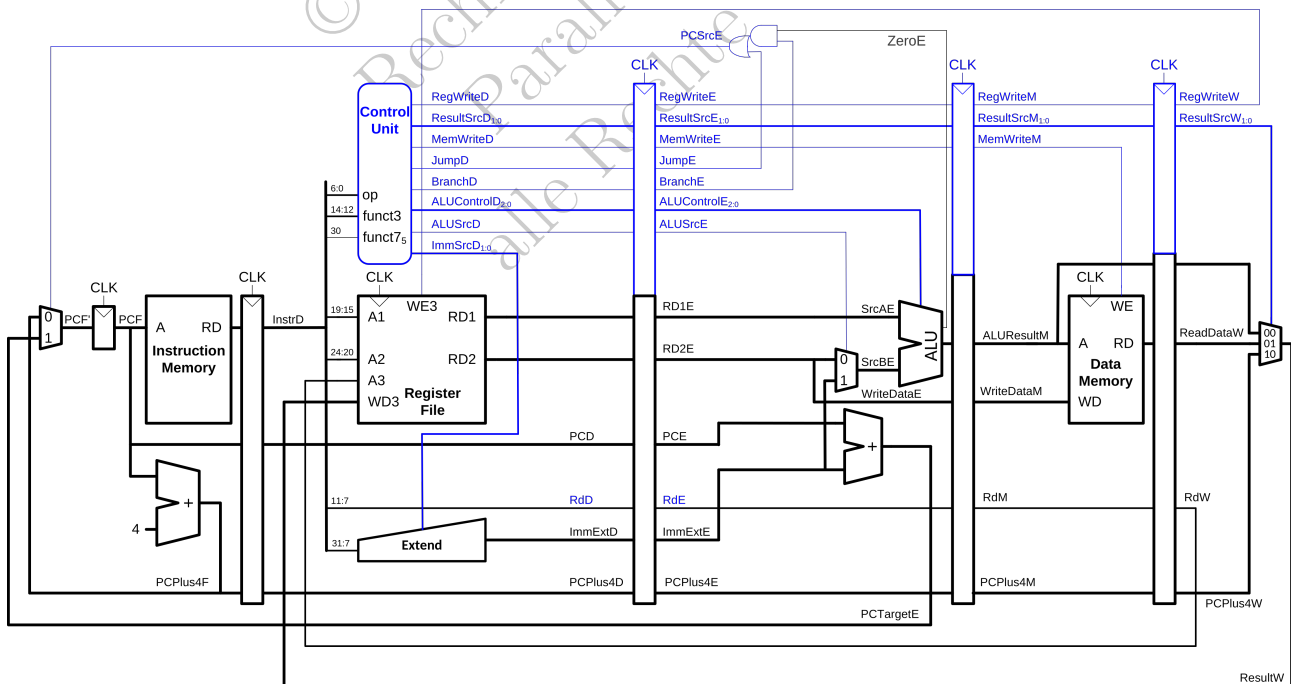
$$\text{Writeback: } (t_{RegRead} + t_{max} + t_{RFSetup}) \cdot 2 = \underline{260ps}$$

$$\text{Taktlänge: } \underline{300ps}$$

2 Flushen der Pipeline (ohne Hazard Unit)

Aus der Vorlesung wissen wir, dass eine Stage der Pipeline geflushed werden kann indem man rücksetzbare Register zwischen Pipelinestages verwendet. In dieser Übung wollen wir Flushing umsetzen *ohne* die Register selbst zu verändern.

Erweitere dazu den RISC-V Prozessor in Abbildung 1, sodass das Flushen der Pipeline bei den Befehlen `beq` und `jal` unterstützt wird. Füge dazu möglichst wenig Logik zum Datenpfad des Prozessors hinzu. Es darf angenommen werden, dass (zusätzlich zu `addi zero, zero, 0`) eine Instruktion die nur aus Nullen besteht, einem NOP entspricht. Bestehende Komponenten dürfen nicht verändert werden.



Setzt diese Änderung in Digital um und verifiziere das Verhalten mit dem beigefügten Testprogramm.

3 Pipelining mit Hazard Unit

Der RISC-V Prozessor mit Pipelining und Hazard Unit aus Abbildung 2 führt den folgenden Programmcode aus.

```
i1:      addi t1, zero, 52
i2:      addi t0, t1, -4
i3:      lw t3, 16(t0)
i4:      sw t3, 20(t0)
i5:      xor t2, t0, t3
i6:      or t2, t2, t3
```

- a) Gebe alle Datenabhängigkeiten im Programm an und klassifiziere diese entsprechend.

Art der Abhängigkeit	Betroffene Befehle	Betroffenes Register
RAW	i1, i2	t1
RAW	i2, i3	t0
RAW	i2, i4	t0
RAW	i2, i5	t0
RAW	i3, i4	t3
RAW	i3, i5	t3
RAW	i3, i6	t3
RAW	i5, i6	t2
WAW	i5, i6	t2

- b) Skizziere den Programmablauf und erkläre welche Konflikte in dem Programm auftreten. Welche davon werden durch Forwarding gelöst werden und von welcher Pipeline-Stufe wird geforwarded? Welche Konflikte werden durch Stalling gelöst?

Hinweis: Beachte, dass der Prozessor Konflikte unmittelbar löst. Die Lösung kann Auswirkungen auf zukünftige Konflikte und deren Lösung haben.

```

i1: addi t1, zero, 52
i2: addi t0, t1, -4
i3: lw t3, 16(t0)
i4: sw t3, 20(t0)
i5: xor t2, t0, t3
i6: or t2, t2, t3

```

Takt	F	D	E	M	W
1	addi				
2	addi	addi			
3	lw	addi	addi		
4	sw	lw	addi	addi	
5	xor	sw	lw	addi	addi
6	xor	sw	NOP	lw	addi
7	or	xor	sw	NOP	lw
8		or	xor	sw	NOP
9			or	xor	sw
10				or	xor
11					or
12					

Konflikt			Konfliktbehebung		
Takt	Betroffene Befehle	Grund	Takt	Art der Behebung	ggf. Pipeline-Stufe
3	i1, i2	RAW t1	4	Forwarding	M → E
4	i2, i3	RAW t0	5	Forwarding	M → E
5	i3, i4	RAW t3	5	Stalling	
6	i3, i4	RAW t3	7	Forwarding	W → E
8	i5, i6	RAW t2	9	Forwarding	M → E

Erinnerung: Der Konflikt tritt in dem Takt auf, in dem (ohne Konfliktbehebung) ein Lesezugriff auf ein Register erfolgt wäre, das noch nicht mit dem semantisch korrekten Wert beschrieben worden wäre. Die Konfliktbehebung erfolgt in dem Takt, in dem das entsprechende Kontrollsignal von der Hazard Unit gesetzt wird.

- c) Mit den Delays aus Tabelle 1 hat der Prozessor aus Abbildung 2 eine Taktlänge von 350ps.

Auf dem Prozessor wird ein Programm ausgeführt das aus 25% lw, 10% sw, 11% beq, 2% jal und 52 % R- oder I-Typ ALU Instruktionen besteht. Nimm an, dass 40% der lw von Befehlen gefolgt werden die das Ergebnis direkt verwenden und 50% der Bran-

ches genommen werden. Bei einer falschen Sprungvorhersage müssen 2 Befehle geflushed werden.

Was ist die vorraussichtliche Laufzeit wenn das Programm aus 10¹¹ Instruktionen besteht? Das initiale Laden der Pipeline kann ignoriert werden.

$$lw: 0,4 \cdot 2 + 0,6 \cdot 1 = \underline{1,4 \text{ Zyklen}}$$

$$jal: \underline{3 \text{ Zyklen}}$$

$$beq: 0,5 \cdot 3 + 0,5 \cdot 1 = \underline{2 \text{ Zyklen}}$$

$$0,25 \cdot 1,4 + 0,1 + 0,11 \cdot 2 + 0,02 \cdot 3 + 0,52 = \underline{1,25}$$
$$10^{11} \cdot 1,25 \cdot 350ps \approx \underline{445}$$

4 Datenabhängigkeiten und Pipeline-Konflikte (Hausaufgabe)

Bearbeitung und Abgabe der Hausaufgabe 12 auf <https://artemis.tum.de/courses/516> bis Sonntag, den 25.01.2026, 23:59 Uhr.

Ziel dieser Übung ist es, ein vorgegebenes Programm durch Einfügen von NOPs und Umordnen der Befehle zu beschleunigen. Betrachte das folgende RISC-V Programm und nimm an, dass es auf einem RISC-V Prozessor mit fünfstufiger Pipeline *ohne Hazard Unit* (siehe Abbildung 1) ausgeführt werden soll. Die Registerbank wird bei steigender Taktflanke geschrieben.

```
i1: add x10, x3, x4
i2: add x1, x4, x5
i3: add x2, x3, x4
i4: addi x7, x0, 3
i5: sub x8, x4, x3
loop:
i6: lw x5, 0(x6)
i7: xor x9, x7, x8
i8: and x3, x3, x4
i9: add x2, x2, x5
i10: addi x10, x10, 1
i11: addi x6, x6, 4
i12: blt x10, x1, loop
i13: sub x10, x6, x1
```

a) Gib alle Datenabhängigkeiten zwischen den Befehlen des Programms an.

Hinweis: Beachte, dass das Programm springen könnte und somit auch Abhängigkeiten zu optisch „früheren“ Befehlen oder sogar optisch „gleichen“ Instruktionen bestehen könnten.